

Adventure Works Financial Analysis Project - Technical (SQL & Python-Pandas)

SQL Syntaxes Used

--Data checking (row count)--

```
SELECT 'FactInternetSales' AS table_name, COUNT(*) AS row_count FROM FactInternetSales
```

```
UNION ALL
```

```
SELECT 'DimProduct', COUNT(*) FROM DimProduct
```

```
UNION ALL
```

```
SELECT 'DimCustomer', COUNT(*) FROM DimCustomer
```

```
UNION ALL
```

```
SELECT 'DimDate', COUNT(*) FROM DimDate
```

```
UNION ALL
```

```
SELECT 'DimGeography', COUNT(*) FROM DimGeography
```

```
UNION ALL
```

```
SELECT 'DimSalesTerritory', COUNT(*) FROM DimSalesTerritory
```

```
UNION ALL
```

```
SELECT 'DimProductSubcategory', COUNT(*) FROM DimProductSubcategory
```

```
UNION ALL
```

```
SELECT 'DimProductCategory', COUNT(*) FROM DimProductCategory
```

```
UNION ALL
```

```
SELECT 'FactSalesQuota', COUNT(*) FROM FactSalesQuota;
```

--checking nulls in key columns--

```
SELECT
```

```
    SUM(CASE WHEN ProductKey IS NULL THEN 1 ELSE 0 END) AS  
missing_product_key,
```

```
    SUM(CASE WHEN CustomerKey IS NULL THEN 1 ELSE 0 END) AS  
missing_customer_key,
```

```
    SUM(CASE WHEN SalesTerritoryKey IS NULL THEN 1 ELSE 0 END) AS  
missing_territory_key,
```

```
SUM(CASE WHEN OrderDateKey IS NULL THEN 1 ELSE 0 END) AS  
missing_order_date_key,  
SUM(CASE WHEN SalesAmount IS NULL THEN 1 ELSE 0 END) AS  
missing_sales_amount,  
SUM(CASE WHEN TotalProductCost IS NULL THEN 1 ELSE 0 END) AS  
missing_total_product_cost  
FROM FactInternetSales;
```

--check duplicate sales rows--

```
SELECT  
    SalesOrderNumber,  
    SalesOrderLineNumber,  
    COUNT(*) AS duplicate_count  
FROM FactInternetSales  
GROUP BY SalesOrderNumber, SalesOrderLineNumber  
HAVING COUNT(*) > 1;
```

--check negative or suspicious values--

```
SELECT  
    COUNT(*) AS suspicious_rows  
FROM FactInternetSales  
WHERE SalesAmount < 0  
    OR TotalProductCost < 0  
    OR OrderQuantity <= 0;
```

--data coverage check--

```
SELECT  
    MIN(OrderDateKey) AS first_order,  
    MAX(OrderDateKey) AS last_order  
FROM FactInternetSales;
```

--checking data integrity--

```
SELECT COUNT(*) AS orphan_products  
FROM FactInternetSales f  
LEFT JOIN DimProduct p  
ON f.ProductKey = p.ProductKey
```

WHERE p.ProductKey IS NULL;

```
SELECT COUNT(*) AS orphan_customers
FROM FactInternetSales f
LEFT JOIN DimCustomer c
ON f.CustomerKey = c.CustomerKey
WHERE c.CustomerKey IS NULL;
```

```
SELECT COUNT(*) AS orphan_territories
FROM FactInternetSales f
LEFT JOIN DimSalesTerritory t
ON f.SalesTerritoryKey = t.SalesTerritoryKey
WHERE t.SalesTerritoryKey IS NULL;
```

```
SELECT COUNT(*) AS orphan_dates
FROM FactInternetSales f
LEFT JOIN DimDate d
ON f.OrderDateKey = d.DateKey
WHERE d.DateKey IS NULL;
```

```
--verify quota data is usable and within ranger--
SELECT
    MIN(SalesAmountQuota) AS min_quota,
    MAX(SalesAmountQuota) AS max_quota,
    AVG(SalesAmountQuota) AS avg_quota
FROM FactSalesQuota;
```

```
SELECT
    COUNT(*) AS null_quotas
FROM FactSalesQuota
WHERE SalesAmountQuota IS NULL;
```

Python Syntaxes Used

Python Syntaxes Used – AdventureWorks Financial Analysis

1. Import Libraries

```
import sqlite3  
import pandas as pd
```

Purpose:

Import SQLite for database connections and pandas for data analysis.

2. Connect to Database

```
conn = sqlite3.connect("adventureworks_financial_analysis.db")
```

Purpose:

Create a connection between Python and the SQLite database.

3. List All Tables in the Database

```
query = """  
SELECT name  
FROM sqlite_master  
WHERE type='table';  
"""
```

```
tables = pd.read_sql(query, conn)
```

tables

Purpose:

Display all tables available in the database.

4. Load a Table into Pandas

```
sales = pd.read_sql("""  
SELECT *  
FROM FactInternetSales  
""", conn)
```

Purpose:

Import the FactInternetSales table into a pandas DataFrame called `sales`.

5. View First Five Rows

```
sales.head()
```

Purpose:

Preview the first five records in the dataset.

6. View Dataset Information

```
sales.info()
```

Purpose:

Display row count, column count, data types, and missing values.

7. Generate Descriptive Statistics

```
sales.describe()
```

Purpose:

Display summary statistics such as count, mean, standard deviation, minimum, maximum, and quartiles.

8. Calculate Total Revenue

```
sales["SalesAmount"].sum()
```

Purpose:

Calculate total sales revenue.

9. Calculate Average Revenue per Order Line

```
round(sales["SalesAmount"].mean(), 2)
```

Purpose:

Calculate the average sales amount and round it to two decimal places.

10. Find Maximum Sales Amount

```
sales["SalesAmount"].max()
```

Purpose:

Identify the highest sales amount recorded.

11. Count Unique Orders

```
sales["SalesOrderNumber"].nunique()
```

Purpose:

Count the number of distinct orders.

12. Count Unique Customers

```
sales["CustomerKey"].nunique()
```

Purpose:

Count the number of distinct customers.

13. Calculate Average Orders per Customer

```
sales["SalesOrderNumber"].nunique() / sales["CustomerKey"].nunique()
```

Purpose:

Calculate the average number of orders placed by each customer.

14. Load Date Dimension Table

```
dates = pd.read_sql("""  
SELECT *  
FROM DimDate  
""", conn)
```

Purpose:

Import the DimDate table into a pandas DataFrame for time-based analysis.

15. Preview Date Table

```
dates.head()
```

Purpose:

Display the first five rows of the DimDate table to verify successful import and inspect available columns.

16. Inspect Date Table Structure

```
dates.info()
```

Purpose:

Review the number of rows, columns, data types, and completeness of the date dimension.

17. Merge Sales and Date Tables

```
sales_dates = sales.merge(  
    dates,  
    left_on="OrderDateKey",  
    right_on="DateKey",  
    how="left"  
)
```


Purpose:

Combine FactInternetSales with DimDate to add year, quarter, month, and other calendar attributes to each sales record.

18. Validate Merge Results

```
sales_dates.shape
```

Purpose:

Confirm that all sales records were retained after the merge and verify the number of columns added.

19. Calculate Revenue by Year

```
revenue_by_year = sales_dates.groupby(  
    "CalendarYear"  
)[["SalesAmount"]].sum()
```

```
revenue_by_year
```

Purpose:

Calculate total revenue generated in each calendar year.

20. Calculate Year-over-Year Revenue Growth

```
round(  
    revenue_by_year.pct_change() * 100,  
    2  
)
```

Purpose:

Measure annual revenue growth rates compared to the previous year.

21. Check Month Coverage by Year

```
sales_dates.groupby(  
    "CalendarYear"  
)[["MonthNumberOfYear"]].agg(  
    ["min", "max", "nunique"]  
)
```

Purpose:

Identify the first month, last month, and number of months available for each year to detect incomplete periods.

22. Calculate Revenue by Year and Month

```
monthly_revenue = sales_dates.groupby(  
    ["CalendarYear", "MonthNumberOfYear"]  
)[["SalesAmount"]].sum()
```

monthly_revenue

Purpose:

Calculate monthly revenue trends and identify seasonal or growth patterns.

23. Calculate Revenue by Year and Quarter

```
quarterly_revenue = sales_dates.groupby(  
    ["CalendarYear", "CalendarQuarter"]
```

```
)["SalesAmount"].sum()
```

quarterly_revenue

Purpose:

Aggregate revenue at the quarterly level for trend analysis and year-over-year comparisons.

24. Convert Quarterly Revenue to Comparison Table

```
quarter_table = sales_dates.groupby(  
    ["CalendarYear", "CalendarQuarter"]  
)["SalesAmount"].sum().unstack()
```

quarter_table

Purpose:

Reshape quarterly revenue data into a matrix format with years as rows and quarters as columns for easier comparison.

25. Compare Quarterly Revenue Growth (2008 vs 2007)

```
round(  
    ((quarter_table.loc[2008] - quarter_table.loc[2007])  
     / quarter_table.loc[2007]) * 100,  
    2  
)
```

Purpose:

Calculate quarter-over-quarter revenue growth between 2008 and the same quarters in 2007.

26. Calculate Annual Revenue Growth (2007 vs 2006)

```
round(  
    (  
        revenue_by_year.loc[2007]  
        - revenue_by_year.loc[2006]  
    )  
    / revenue_by_year.loc[2006]  
    * 100,  
    2  
)
```

Purpose:

Calculate full-year revenue growth using only complete years of data.

27. Calculate Orders by Quarter

```
orders_by_quarter = sales_dates.groupby(  
    ["CalendarYear", "CalendarQuarter"]  
)["SalesOrderNumber"].nunique()
```

orders_by_quarter

Purpose:

Count unique orders in each quarter to analyze changes in transaction volume.

28. Convert Quarterly Orders to Comparison Table

```
orders_table = sales_dates.groupby(  
    ["CalendarYear", "CalendarQuarter"]  
)["SalesOrderNumber"].nunique()
```

```
["CalendarYear", "CalendarQuarter"]  
)["SalesOrderNumber"].nunique().unstack()
```

orders_table

Purpose:

Reshape quarterly order counts into a table for quarter-by-quarter comparisons.

29. Compare Quarterly Order Growth (2008 vs 2007)

```
round(  
    (  
        orders_table.loc[2008]  
        - orders_table.loc[2007]  
    )  
    / orders_table.loc[2007]  
    * 100,  
    2  
)
```

Purpose:

Measure growth in order volume between matching quarters of 2007 and 2008.

30. Calculate Unique Customers by Year

```
sales_dates.groupby(  
    "CalendarYear"  
)["CustomerKey"].nunique()
```

Purpose:

Count unique customers in each year to evaluate customer acquisition trends.

31. Calculate Average Order Value (AOV) by Quarter

```
aov_by_quarter = (  
    sales_dates.groupby(  
        ["CalendarYear", "CalendarQuarter"]  
    )["SalesAmount"].sum()  
    /  
    sales_dates.groupby(  
        ["CalendarYear", "CalendarQuarter"]  
    )["SalesOrderNumber"].nunique()  
)
```

aov_by_quarter

Purpose:

Calculate the average revenue generated per order in each quarter.

32. Convert AOV to Comparison Table

```
aov_table = aov_by_quarter.unstack()
```

aov_table

Purpose:

Reshape Average Order Value data into a year-versus-quarter comparison table for trend analysis.

33. Calculate total revenue by product

```
top_products_revenue = (  
    sales_dates.groupby("ProductKey")["SalesAmount"]  
    .sum()  
    .sort_values(ascending=False)  
    .reset_index()  
)
```

top_products_revenue

Purpose:

Calculated total sales revenue for each product to identify the highest revenue-generating products.

34. Load product details table

```
products = pd.read_sql("""  
SELECT *  
FROM DimProduct  
""", conn)
```

```
products.head()
```

Purpose:

Loaded the product dimension table so product names could be added to the revenue and profit results.

35. Check product table columns

```
list(products.columns)
```

Purpose:

Checked available product columns and confirmed that **ProductKey** and **EnglishProductName** could be used for merging.

36. Add product names to top revenue products

```
top_products_named = top_products_revenue.merge(  
    products[["ProductKey", "EnglishProductName"]],  
    on="ProductKey",  
    how="left"  
)
```

```
top_products_named.head(10)
```

Purpose:

Merged product revenue results with product names to make the output easier to interpret.

37. Calculate profit by product

```
top_products_profit = (  
    sales_dates.groupby("ProductKey")["Profit"]  
    .sum()  
    .sort_values(ascending=False)  
    .reset_index()  
)
```

```
top_products_profit
```

Purpose:

Calculated total profit for each product to identify the most profitable products.

38. Add product names to top profit products

```
top_products_profit_named = top_products_profit.merge(  
    products[["ProductKey", "EnglishProductName"]],  
    on="ProductKey",  
    how="left"
```


)

```
top_products_profit_named.head(10)
```

Purpose:

Merged product profit results with product names to identify the most profitable products by name.

39. Calculate product profit margin

```
products_margin = (  
    sales_dates.groupby("ProductKey")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum"  
    })  
    .reset_index()  
)
```

```
products_margin["ProfitMargin"] = (  
    products_margin["Profit"] / products_margin["SalesAmount"]  
)
```

```
products_margin = products_margin.merge(  
    products[["ProductKey", "EnglishProductName"]],  
    on="ProductKey",  
    how="left"  
)
```

```
products_margin.sort_values("ProfitMargin", ascending=False)
```

Purpose:

Calculated profit margin for each product to identify which products were most efficient at turning revenue into profit.

40. Filter meaningful high-margin products

```
top_margin_products = (  
    products_margin[products_margin["SalesAmount"] > 50000]  
    .sort_values("ProfitMargin", ascending=False)  
)
```

```
top_margin_products
```

Purpose:

Filtered out very low-revenue products so the margin analysis focused on products with meaningful sales volume.

41. Compare revenue and margin by product

```
revenue_margin = (  
    products_margin  
    .sort_values("SalesAmount", ascending=False)  
)
```

```
revenue_margin.head(20)
```

Purpose:

Compared product revenue and profit margin together to identify high-revenue products with lower margins.

42. Load product subcategory table

```
subcategory = pd.read_sql("""  
SELECT *  
FROM DimProductSubcategory  
""", conn)
```

```
subcategory.head()
```

Purpose:

Loaded product subcategory details so products could be grouped by subcategory.

43. Load product category table

```
categories = pd.read_sql("""  
SELECT *  
FROM DimProductCategory  
""", conn)
```

```
categories.head()
```

Purpose:

Loaded product category details so product performance could be analyzed at the category level.

44. Convert merge key data types

```
products["ProductSubcategoryKey"] =  
products["ProductSubcategoryKey"].astype("str")  
subcategory["ProductSubcategoryKey"] =  
subcategory["ProductSubcategoryKey"].astype("str")
```

```
subcategory["ProductCategoryKey"] =  
subcategory["ProductCategoryKey"].astype("str")  
categories["ProductCategoryKey"] =  
categories["ProductCategoryKey"].astype("str")
```

Purpose:

Converted key columns to the same data type so product, subcategory, and category tables could be merged correctly.

45. Keep only sold products

```
sold_products = products[
    products["ProductKey"].isin(sales_dates["ProductKey"].unique())
][["ProductKey", "ProductSubcategoryKey", "EnglishProductName"]]

sold_products.head()
```

Purpose:

Filtered the product table to include only products that appeared in the sales data.

46. Merge sales data with product names and subcategories

```
sales_product_category = sales_dates.merge(
    sold_products,
    on="ProductKey",
    how="left"
)
```

```
sales_product_category.head()
```

Purpose:

Added product names and subcategory keys to the sales data.

47. Merge subcategory names into sales data

```
sales_product_category = sales_product_category.merge(
    subcategory[
        "ProductSubcategoryKey",
        "ProductCategoryKey",
        "EnglishProductSubcategoryName"
    ]
```

```
    ]  
  ],  
  on="ProductSubcategoryKey",  
  how="left"  
)
```

```
sales_product_category.head()
```

Purpose:

Added product subcategory names and category keys to the sales dataset.

48. Merge category names into sales data

```
sales_product_category = sales_product_category.merge(  
  categories[  
    [  
      "ProductCategoryKey",  
      "EnglishProductCategoryName"  
    ]  
  ],  
  on="ProductCategoryKey",  
  how="left"  
)
```

```
sales_product_category.head()
```

Purpose:

Added product category names to complete the product-category analysis table.

49. Calculate category profitability

```
category_profitability = (  
  sales_product_category
```

```

.groupby("EnglishProductCategoryName")
.agg({
    "SalesAmount": "sum",
    "Profit": "sum"
})
.reset_index()
)

```

```

category_profitability["ProfitMargin"] = (
    category_profitability["Profit"]
    / category_profitability["SalesAmount"]
)

```

```

category_profitability.sort_values("Profit", ascending=False)

```

Purpose:

Calculated revenue, profit, and profit margin by product category to identify which categories drive profitability.

50. Calculate subcategory profitability

```

subcategory_profitability = (
    sales_product_category
    .groupby("EnglishProductSubcategoryName")
    .agg({
        "SalesAmount": "sum",
        "Profit": "sum"
    })
    .reset_index()
)

```

```

subcategory_profitability["ProfitMargin"] = (
    subcategory_profitability["Profit"]
    / subcategory_profitability["SalesAmount"]
)

```

)

```
subcategory_profitability.sort_values("Profit", ascending=False)
```

Purpose:

Calculated revenue, profit, and profit margin by product subcategory to identify the strongest and weakest product groups.

51. Load Sales Territory Table

```
territories = pd.read_sql("""  
SELECT *  
FROM DimSalesTerritory  
""", conn)
```

```
territories.head()
```

Purpose:

Loaded the sales territory dimension table to access region, country, and territory group information.

52. Merge Territory Data with Sales Data

```
sales_region = sales_dates.merge(  
    territories[  
        [  
            "SalesTerritoryKey",  
            "SalesTerritoryRegion",  
            "SalesTerritoryCountry",  
            "SalesTerritoryGroup"  
        ]  
    ],  
    on="SalesTerritoryKey",  
    how="left"  
)
```

```
sales_region.head()
```

Purpose:

Merged sales transactions with territory information to create the primary dataset for regional analysis.

53. Calculate Revenue and Profit by Region

```
region_performance = (  
    sales_region  
    .groupby("SalesTerritoryRegion")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum"  
    })  
    .reset_index()  
    .sort_values("SalesAmount", ascending=False)  
)
```

```
region_performance
```

Purpose:

Calculated total revenue and profit for each region to identify the strongest-performing territories.

54. Calculate Regional Profitability and Cost Efficiency

```
region_cost_performance = (  
    sales_region  
    .groupby("SalesTerritoryRegion")  
    .agg({
```



```

        "SalesAmount": "sum",
        "TotalProductCost": "sum",
        "Profit": "sum"
    })
    .reset_index()
)

region_cost_performance["ProfitMarginPct"] = (
    region_cost_performance["Profit"]
    / region_cost_performance["SalesAmount"]
) * 100

region_cost_performance.sort_values(
    "ProfitMarginPct",
    ascending=True
)

```

Purpose:

Calculated revenue, cost, profit, and profit margin by region to evaluate cost efficiency and profitability.

55. Calculate Revenue per Customer and Profit per Order

```

region_customer_order = (
    sales_region
    .groupby("SalesTerritoryRegion")
    .agg({
        "SalesAmount": "sum",
        "Profit": "sum",
        "CustomerKey": "nunique",
        "SalesOrderNumber": "nunique"
    })
    .reset_index()
)

```

```

)

region_customer_order["RevenuePerCustomer"] = (
    region_customer_order["SalesAmount"]
    / region_customer_order["CustomerKey"]
)

region_customer_order["ProfitPerOrder"] = (
    region_customer_order["Profit"]
    / region_customer_order["SalesOrderNumber"]
)

region_customer_order.sort_values(
    "RevenuePerCustomer",
    ascending=False
)

```

Purpose:

Measured customer value and order profitability by region using revenue per customer and profit per order metrics.

56. Merge Territory Information into Product Analysis Table

```

sales_product_region = sales_product_category.merge(
    territories[
        [
            "SalesTerritoryKey",
            "SalesTerritoryRegion",
            "SalesTerritoryCountry",
            "SalesTerritoryGroup"
        ]
    ],
    on="SalesTerritoryKey",

```

```
        how="left"  
    )  
  
sales_product_region.head()
```

Purpose:

Combined product, sales, and regional information to support regional product-category analysis.

57. Analyze Product Category Performance by Region

```
region_category_performance = (  
    sales_product_region  
    .groupby(  
        [  
            "SalesTerritoryRegion",  
            "EnglishProductCategoryName"  
        ]  
    )  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum"  
    })  
    .reset_index()  
    .sort_values("SalesAmount", ascending=False)  
)
```

```
region_category_performance.head(20)
```

Purpose:

Calculated revenue and profit by product category within each region to identify regional product preferences.

58. Create Region–Category Comparison Table

```
region_category_table = region_category_performance.pivot(
    index="SalesTerritoryRegion",
    columns="EnglishProductCategoryName",
    values="SalesAmount"
).fillna(0)
```

region_category_table

Purpose:

Created a comparison matrix showing category revenue across regions to identify differences in category performance.

59. Calculate Revenue Growth by Region and Year

```
region_year_growth = (
    sales_region
    .groupby(
        [
            "CalendarYear",
            "SalesTerritoryRegion"
        ]
    )
    .agg({
        "SalesAmount": "sum",
        "Profit": "sum"
    })
    .reset_index()
)
```

region_year_growth

Purpose:

Calculated annual revenue and profit by region to analyze regional growth trends over time.

60. Load Customer Table

```
customers = pd.read_sql("""  
SELECT *  
FROM DimCustomer  
""", conn)
```

```
customers.head()
```

Purpose:

Loaded the customer dimension table to access customer demographic information such as gender, birth date, yearly income, education, occupation, marital status, and commute distance.

61. Review Customer Columns

```
customers.columns.tolist()
```

Purpose:

Reviewed the available customer fields to determine which demographic variables could support customer analysis.

62. Merge Customer Data with Sales Data

```
customer_sales = sales_region.merge(  
    customers,  
    on="CustomerKey",  
    how="left"  
)
```

```
customer_sales.head()
```

Purpose:

Merged customer demographic data with sales and regional transaction data using **CustomerKey**.

63. Verify Customer-Sales Merge

```
customer_sales.shape
```

Purpose:

Checked the size of the merged customer-sales dataset to confirm that the merge worked correctly.

64. Preview Key Customer-Sales Fields

```
customer_sales[
    [
        "CustomerKey",
        "SalesAmount",
        "Profit",
        "Gender",
        "EnglishOccupation"
    ]
].head()
```

Purpose:

Confirmed that sales metrics and customer demographic fields were successfully combined in the same dataset.

65. Calculate Top Customers by Revenue

```
top_customers = (  
    customer_sales  
    .groupby("CustomerKey")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum",  
        "SalesOrderNumber": "nunique"  
    })  
    .reset_index()  
    .sort_values("SalesAmount", ascending=False)  
)  
  
top_customers.head(10)
```

Purpose:

Identified the top customers by total revenue, total profit, and number of unique orders.

66. Add Customer Names and Demographics to Top Customers

```
top_customers_named = (  
    customer_sales  
    .groupby([  
        "CustomerKey",  
        "FirstName",  
        "LastName",  
        "EnglishOccupation",  
        "YearlyIncome"  
    ])  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum",  
    })  
)
```

```
        "SalesOrderNumber": "nunique"
    })
    .reset_index()
    .sort_values("SalesAmount", ascending=False)
)

top_customers_named.head(10)
```

Purpose:

Added customer names, occupations, and income levels to make top-customer results easier to interpret.

67. Calculate Average Order Value for Top Customers

```
top_customers_named["AvgOrderValue"] = (
    top_customers_named["SalesAmount"]
    / top_customers_named["SalesOrderNumber"]
)

top_customers_named.head(10)
```

Purpose:

Calculated average order value for top customers to understand whether high revenue came from larger orders or repeat purchases.

68. Analyze Customer Value by Occupation

```
occupation_performance = (
    customer_sales
    .groupby("EnglishOccupation")
    .agg({
        "SalesAmount": "sum",
```



```

        "Profit":"sum",
        "CustomerKey":"nunique",
        "SalesOrderNumber":"nunique"
    })
    .reset_index()
    .sort_values("SalesAmount", ascending=False)
)

```

occupation_performance

Purpose:

Calculated total revenue, total profit, unique customers, and orders by occupation segment.

69. Calculate Revenue and Profit per Customer by Occupation

```

occupation_performance["RevenuePerCustomer"] = (
    occupation_performance["SalesAmount"]
    / occupation_performance["CustomerKey"]
)

```

```

occupation_performance["ProfitPerCustomer"] = (
    occupation_performance["Profit"]
    / occupation_performance["CustomerKey"]
)

```

```

occupation_performance.sort_values(
    "RevenuePerCustomer",
    ascending=False
)

```

Purpose:

Normalized occupation performance by customer count to identify which occupation segments generated the highest customer value.

70. Create Customer Age from Birth Date

```
customer_sales["BirthDate"] = pd.to_datetime(customer_sales["BirthDate"])
```

```
customer_sales["Age"] = (  
    customer_sales["CalendarYear"]  
    - customer_sales["BirthDate"].dt.year  
)
```

```
customer_sales["Age"].describe()
```

Purpose:

Calculated customer age at the time of purchase using transaction year and birth date, then reviewed the age distribution.

71. Create Age Groups

```
customer_sales["AgeGroup"] = pd.cut(  
    customer_sales["Age"],  
    bins=[20, 29, 39, 49, 59, 100],  
    labels=[  
        "21-29",  
        "30-39",  
        "40-49",  
        "50-59",  
        "60+"  
    ]  
)
```

Purpose:

Created age segments based on the actual age range in the dataset.

72. Analyze Revenue and Profit by Age Group

```
age_performance = (  
    customer_sales  
    .groupby("AgeGroup")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum",  
        "CustomerKey": "nunique",  
        "SalesOrderNumber": "nunique"  
    })  
    .reset_index()  
)
```

age_performance

Purpose:

Calculated total revenue, profit, unique customers, and orders for each age group.

73. Calculate Revenue and Profit per Customer by Age Group

```
age_performance["RevenuePerCustomer"] = (  
    age_performance["SalesAmount"]  
    / age_performance["CustomerKey"]  
)
```

```
age_performance["ProfitPerCustomer"] = (  
    age_performance["Profit"]  
    / age_performance["CustomerKey"]
```

```
)  
  
age_performance.sort_values(  
    "RevenuePerCustomer",  
    ascending=False  
)
```

Purpose:

Identified which age groups generated the highest revenue and profit per customer.

74. Review Yearly Income Distribution

```
customer_sales["YearlyIncome"].describe()
```

Purpose:

Reviewed income distribution to create meaningful income segments based on the dataset.

75. Create Income Groups

```
customer_sales["IncomeGroup"] = pd.cut(  
    customer_sales["YearlyIncome"],  
    bins=[0, 30000, 60000, 90000, 120000, 200000],  
    labels=[  
        "Low Income (<30K)",  
        "Lower-Middle (30K-60K)",  
        "Middle (60K-90K)",  
        "Upper-Middle (90K-120K)",  
        "High Income (120K+)"  
    ]  
)
```

Purpose:

Created income segments to analyze customer value by yearly income range.

76. Check Income Group Distribution

```
customer_sales["IncomeGroup"].value_counts().sort_index()
```

Purpose:

Checked how many sales records belonged to each income segment.

77. Analyze Revenue and Profit by Income Group

```
income_performance = (  
    customer_sales  
    .groupby("IncomeGroup")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum",  
        "CustomerKey": "nunique",  
        "SalesOrderNumber": "nunique"  
    })  
    .reset_index()  
)
```

```
income_performance
```

Purpose:

Calculated total revenue, profit, unique customers, and orders for each income group.

78. Calculate Revenue and Profit per Customer by Income Group

```
income_performance["RevenuePerCustomer"] = (  
    income_performance["SalesAmount"]  
    / income_performance["CustomerKey"]  
)
```

```
income_performance["ProfitPerCustomer"] = (  
    income_performance["Profit"]  
    / income_performance["CustomerKey"]  
)
```

```
income_performance.sort_values(  
    "RevenuePerCustomer",  
    ascending=False  
)
```

Purpose:

Measured customer value by income group and identified the relationship between income and revenue/profit per customer.

79. Analyze Gender Performance

```
gender_performance = (  
    customer_sales  
    .groupby("Gender")  
    .agg({  
        "SalesAmount": "sum",  
        "Profit": "sum",  
        "CustomerKey": "nunique",  
        "SalesOrderNumber": "nunique"  
    })  
    .reset_index()
```

)

gender_performance

Purpose:

Calculated total revenue, profit, unique customers, and orders by gender.

80. Calculate Revenue and Profit per Customer by Gender

```
gender_performance["RevenuePerCustomer"] = (  
    gender_performance["SalesAmount"]  
    / gender_performance["CustomerKey"]  
)
```

```
gender_performance["ProfitPerCustomer"] = (  
    gender_performance["Profit"]  
    / gender_performance["CustomerKey"]  
)
```

```
gender_performance.sort_values(  
    "RevenuePerCustomer",  
    ascending=False  
)
```

Purpose:

Measured whether gender was meaningfully associated with customer value.

81. Analyze Income Segment Purchase Behavior

```
income_behavior = (  
    customer_sales
```

```

.groupby("IncomeGroup")
.agg({
    "SalesAmount":"sum",
    "SalesOrderNumber":"nunique",
    "CustomerKey":"nunique"
})
.reset_index()
)

income_behavior["AvgOrderValue"] = (
    income_behavior["SalesAmount"]
    / income_behavior["SalesOrderNumber"]
)

income_behavior["PurchaseFrequency"] = (
    income_behavior["SalesOrderNumber"]
    / income_behavior["CustomerKey"]
)

income_behavior.sort_values(
    "AvgOrderValue",
    ascending=False
)

```

Purpose:

Calculated average order value and purchase frequency by income group to determine whether higher-income customers were more valuable due to larger orders, more frequent orders, or both.

82. Analyze Age Group Purchase Behavior

```

age_behavior = (
    customer_sales

```



```

.groupby("AgeGroup")
.agg({
    "SalesAmount":"sum",
    "SalesOrderNumber":"nunique",
    "CustomerKey":"nunique"
})
.reset_index()
)

age_behavior["AvgOrderValue"] = (
    age_behavior["SalesAmount"]
    / age_behavior["SalesOrderNumber"]
)

age_behavior["PurchaseFrequency"] = (
    age_behavior["SalesOrderNumber"]
    / age_behavior["CustomerKey"]
)

age_behavior.sort_values(
    "AvgOrderValue",
    ascending=False
)

```

Purpose:

Calculated average order value and purchase frequency by age group to understand why the 30–39 and 40–49 age groups generated higher customer value.

83. Analyze Occupation Purchase Behavior

```

occupation_behavior = (
    customer_sales

```

```

.groupby("EnglishOccupation")
.agg({
    "SalesAmount":"sum",
    "Profit":"sum",
    "CustomerKey":"nunique",
    "SalesOrderNumber":"nunique"
})
.reset_index()
)

occupation_behavior["RevenuePerCustomer"] = (
    occupation_behavior["SalesAmount"]
    / occupation_behavior["CustomerKey"]
)

occupation_behavior["ProfitPerCustomer"] = (
    occupation_behavior["Profit"]
    / occupation_behavior["CustomerKey"]
)

occupation_behavior["PurchaseFrequency"] = (
    occupation_behavior["SalesOrderNumber"]
    / occupation_behavior["CustomerKey"]
)

occupation_behavior.sort_values(
    "RevenuePerCustomer",
    ascending=False
)

```

Purpose:

Compared occupation segments using revenue per customer, profit per customer, and purchase frequency to determine whether occupation was strongly associated with high-value customers.

84. Dashboard Strategic Recommendations Redesign

```
function Tab8() {
  const [openGroup, setOpenGroup] = useState("01");

  const groups = [
    {
      num: "01",
      title: "Product Strategy",
      subtitle: "Protect core product lines and reduce weak category drag.",
      color: C.accent,
      visual: "🚲",
      metrics: ["Road + Mountain Bikes", "~$10M+ profit", "Mountain-200 / Road-150"],
      impact: ["Higher total profit", "Core product focus", "Lower underperformer cost"],
      items: [
        { action: "Protect & grow Road Bikes and Mountain Bikes", detail: "These two sub-categories generate ~$10M+ in combined profit. Prioritise inventory, marketing spend, and product development here." },
        { action: "Prioritise Mountain-200 and Road-150 product lines", detail: "Top-performing individual lines. Ensure consistent availability, strong merchandising, and upsell/cross-sell placement." },
        { action: "Review or reduce the Clothing category", detail: "Only ~$0.34M revenue and ~$0.14M profit. Resources may generate higher returns if redirected to Accessories or core Bike lines." },
      ],
    },
    {
      num: "02",
      title: "Profitability Strategy",
      subtitle: "Increase profit per order through accessories and smarter pricing.",
    }
  ]
}
```

color: C.amber,
visual: "🛡️ + 🚲",
metrics: ["~62.6% accessory margin", "+\$63 gross profit per \$100 bundle", "Track AOV & profit/order"],
impact: ["Higher AOV", "Better profit/order", "More margin-efficient growth"],
items: [
 { action: "Increase accessory attachment rates at point of sale", detail: "Each accessory sold alongside a bike adds high-margin revenue without proportional cost increase." },
 { action: "Bundle helmets, tires & tubes, hydration packs, bike racks, stands, and bottles with bike purchases", detail: "Accessories carry ~62.6% margins. A \$100 accessory bundle adds ~\$63 in gross profit vs ~\$40 from an equivalent bike revenue increase." },
 { action: "Track AOV and profit per order as primary KPIs", detail: "Revenue growth alone is misleading if AOV continues to decline. Shift focus to value quality, not just volume." },
],
},
{
 num: "03",
 title: "Regional Growth Strategy",
 subtitle: "Defend strong regions and expand high-potential markets.",
 color: C.blue,
 visual: "🌍",
 metrics: ["Australia = anchor market", "Southwest = growth region", "UK & Germany = premium potential"],
 impact: ["Stronger market leadership", "Higher regional value", "Sustainable revenue growth"],
 items: [
 { action: "Prioritise Australia — protect and deepen the relationship", detail: "Highest revenue, profit, revenue per customer, and profit per order. Australia is the anchor market and must be defended." },

{ action: "Accelerate Southwest expansion", detail: "Second-largest region with strong growth trajectory. Invest in customer acquisition and fulfilment capacity here." },

{ action: "Use UK and Germany for premium-product and accessory campaigns", detail: "Both regions show above-average customer value, suggesting receptivity to premium offers and accessory bundles." },

],

},

{

num: "04",

title: "Customer Strategy",

subtitle: "Target highest-value customer segments and improve lifetime value.",

color: C.purple,

visual: "👥",

metrics: ["Age 30–49", "High income", "Professional / Management"],

impact: ["Higher revenue/customer", "More repeat purchases", "Higher lifetime value"],

items: [

{ action: "Target high-income customers aged 30–49 as the primary acquisition segment", detail: "This cohort generates ~2× the revenue per customer of the lowest-income segment and has higher AOV and purchase frequency." },

{ action: "Prioritise Professional and Management occupations in digital and direct campaigns", detail: "Revenue per customer: Professional ~\$1,795, Management ~\$1,778 — well above Manual at ~\$1,199." },

{ action: "Avoid gender as a primary targeting variable", detail: "Gender explains only ~4% variance in customer value. Gender-based campaigns are unlikely to justify the investment." },

],

},

{

num: "05",

title: "Risk Mitigation Strategy",

```
    subtitle: "Reduce concentration risk and build a more balanced
business.",
    color: C.red,
    visual: "⚠️",
    metrics: ["Reduce Bikes dependence", "Diversify beyond Australia",
"Reverse 73.8% AOV decline"],
    impact: ["Lower business risk", "More balanced revenue mix", "Stronger
sustainable growth"],
    items: [
      { action: "Reduce dependence on Bikes and Australia through
deliberate category diversification", detail: "No single product category or
geography should account for >70% of revenue. Set a 3-year diversification
target." },
      { action: "Grow Accessories as both a margin and diversification
strategy", detail: "At ~62.6% margins, growing Accessories from $0.7M to
$2M+ would add ~$800K in gross profit and reduce concentration risk." },
      { action: "Improve AOV through bundles, tiered discounts, and
cross-selling", detail: "AOV declined 73.8% over the analysis period.
Bundles, minimum-spend incentives, and loyalty programmes can reverse
this trend." },
    ],
  },
];
```

```
return (
  <div style={{ display: "flex", flexDirection: "column", gap: 20 }}>
    <div style={{ display: "grid", gridTemplateColumns: "1.4fr 1fr", gap: 18
}}>
      <div style={{ background: C.card, border: `1px solid ${C.border}`,
borderRadius: 12, padding: 24 }}>
        <div style={{ fontSize: 26, fontWeight: 800, color: C.textPrimary,
marginBottom: 8 }}>
          Strategic Recommendations
        </div>
        <div style={{ fontSize: 14, color: C.textSecondary, lineHeight: 1.6 }}>
```

Interactive executive action plan focused on sustainable growth, profitability improvement, customer value, and risk reduction.

</div>

</div>

<div style={{ background: `\${C.accent}12`, border: `1px solid \${C.accent}55`, borderRadius: 12, padding: 20 }}>

<div style={{ fontSize: 13, fontWeight: 800, color: C.accent, marginBottom: 8 }}>

 Our Goal

</div>

<div style={{ fontSize: 14, color: C.textPrimary, lineHeight: 1.6 }}>

Build a more profitable, higher-value, and more balanced
AdventureWorks business.

</div>

</div>

</div>

```
{groups.map((g) => {  
  const isOpen = openGroup === g.num;
```

```
  return (  
    <div
```

```
      key={g.num}
```

```
      onClick={() => setOpenGroup(isOpen ? "" : g.num)}  
      style={{
```

```
        background: C.card,
```

```
        border: `1px solid ${isOpen ? g.color : C.border}`,
```

```
        borderRadius: 14,
```

```
        overflow: "hidden",
```

```
        cursor: "pointer",
```

```
        boxShadow: isOpen ? `0 0 0 1px ${g.color}33, 0 18px 40px
```

```
        rgba(0,0,0,0.25)` : "none",  
        transition: "all 0.25s ease",  
      }}  
    )  
  )  
}
```

```
  )  
}
```

```
  )  
}
```

```

>
  <div style={{ display: "grid", gridTemplateColumns: "1.1fr 1fr 1fr",
gap: 20, padding: 22 }}>
    <div>
      <div style={{ display: "flex", alignItems: "center", gap: 14,
marginBottom: 12 }}>
        <div style={{
          width: 44,
          height: 44,
          borderRadius: 12,
          background: `${g.color}22`,
          color: g.color,
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          fontSize: 18,
          fontWeight: 900,
        }}>
          {g.num}
        </div>
        <div>
          <div style={{ fontSize: 20, fontWeight: 800, color: C.textPrimary
}}>{g.title}</div>
          <div style={{ fontSize: 13, color: C.textSecondary, marginTop:
4 }}>{g.subtitle}</div>
        </div>
      </div>

      <div style={{ fontSize: 54, margin: "22px 0 14px"
}}>{g.visual}</div>

    <div style={{ display: "grid", gridTemplateColumns: "repeat(3,
1fr)", gap: 8 }}>
      {g.metrics.map((m, i) => (
        <div key={i} style={{

```



```

        border: `1px solid ${g.color}44`,
        background: `${g.color}10`,
        borderRadius: 10,
        padding: "10px 8px",
        textAlign: "center",
        fontSize: 12,
        color: C.textPrimary,
        minHeight: 48,
      }}>
      {m}
    </div>
  )))
</div>
</div>

```

```

<div style={{ borderLeft: `1px solid ${C.border}`, paddingLeft: 20
}}>

```

```

  <div style={{ color: g.color, fontSize: 14, fontWeight: 800,
marginBottom: 14 }}>

```

Key Actions

```

</div>

```

```

<div style={{ display: "flex", flexDirection: "column", gap: 12 }}>

```

```

  {g.items.map((item, i) => (

```

```

    <div key={i} style={{ display: "flex", gap: 10 }}>

```

```

      <div style={{

```

```

        width: 18,

```

```

        height: 18,

```

```

        minWidth: 18,

```

```

        borderRadius: "50%",

```

```

        background: g.color,

```

```

        color: C.bg,

```

```

        fontSize: 11,

```

```

        fontWeight: 900,

```

```

        display: "flex",

```

```

        alignItems: "center",

```

```

        justifyContent: "center",
        marginTop: 1,
    }}>
    ✓
</div>
<div>
    <div style={{ fontSize: 13, fontWeight: 700, color:
C.textPrimary, lineHeight: 1.45 }}>
        {item.action}
    </div>
    {isOpen && (
        <div style={{ fontSize: 12, color: C.textSecondary,
lineHeight: 1.6, marginTop: 5 }}>
            {item.detail}
        </div>
    )}
    </div>
</div>
    )}}
</div>
</div>

<div style={{ borderLeft: `1px solid ${C.border}`, paddingLeft: 20
}}>
    <div style={{ color: g.color, fontSize: 14, fontWeight: 800,
marginBottom: 14 }}>
        Expected Impact
    </div>
    <div style={{ display: "flex", flexDirection: "column", gap: 14 }}>
        {g.impact.map((impact, i) => (
            <div key={i} style={{
                display: "flex",
                alignItems: "center",
                gap: 12,

```

```

borderBottom: i < g.impact.length - 1 ? `1px solid ${C.border}`
: "none",
paddingBottom: i < g.impact.length - 1 ? 12 : 0,
}}>
<div style={{ fontSize: 26 }}>
  {i === 0 ? "📊" : i === 1 ? "🎯" : "💰"}
</div>
<div style={{ fontSize: 13, color: C.textPrimary, lineHeight:
1.45 }}>
  {impact}
</div>
</div>
)}}
</div>
</div>

<div style={{
borderTop: `1px solid ${C.border}`,
background: `${g.color}10`,
padding: "10px 22px",
fontSize: 12,
color: C.textSecondary,
display: "flex",
justifyContent: "space-between",
}}>
  <span>{isOpen ? "Click to collapse details" : "Click to expand
evidence and details"}</span>
  <span style={{ color: g.color, fontWeight: 800 }}>{isOpen ? "-" :
"+"}</span>
</div>
</div>
);
}}

```

```
<div style={{ fontSize: 12, color: C.textMuted, lineHeight: 1.6 }}>
  All values are approximate (~) unless stated otherwise. 2005 and
  2008 are partial-year periods.
</div>
</div>
);
}
```

Purpose:

Created an interactive Strategic Recommendations dashboard section using React. The syntax uses `useState()` to control expandable recommendation cards, `map()` to dynamically generate strategy sections, and event handlers (`onClick`) to allow users to expand and collapse recommendation details. This improved dashboard usability while preserving all business insights, KPIs, findings, and strategic recommendations from the financial analysis project.